

SOLID Principles in Java – Designing Enterprise-Level Banking Systems

1. From Classes to Enterprise Systems

So far, students have built an `Account` class in Java such as:

```
public class Account {  
  
    public void deposit(double amount) {  
        // ...  
    }  
  
    public void withdraw(double amount) {  
        // ...  
    }  
}
```

This works well when the application is small.

But imagine the banking system grows. We now need:

- Savings Accounts
- Current Accounts
- Salary Accounts
- Fixed Deposits
- Loan Accounts
- Different withdrawal rules
- Different interest calculations
- PIN authentication
- OTP authentication
- Transaction logging
- Email notifications
- SMS notifications
- Database storage
- Fraud detection
- Audit trails
- Different payment methods

If everything keeps going into one or two classes, the code becomes difficult to maintain.

```
public class Bank {  
  
    public void createAccount() { }  
    public void calculateInterest() { }  
    public void sendEmail() { }  
    public void sendSms() { }  
    public void validatePin() { }  
    public void saveToDatabase() { }  
    public void generateReport() { }  
    public void detectFraud() { }  
}
```

This class is becoming responsible for **too many things**. A small change in one feature can break another.

This is where **SOLID principles** become important.

2. What is SOLID?

SOLID is a set of five object-oriented design principles that help build software that is maintainable, flexible, testable, reusable, extensible, and loosely coupled.

Letter	Principle	Main Idea
S	Single Responsibility Principle	A class should have one reason to change
O	Open/Closed Principle	Open for extension, closed for modification
L	Liskov Substitution Principle	Subtypes must be substitutable for their base types
I	Interface Segregation Principle	Don't force classes to depend on methods they don't use
D	Dependency Inversion Principle	Depend on abstractions, not concrete implementations

A useful way to remember them:

S – One job

O – Add without breaking

L – Child should behave like parent

I – Keep interfaces small

D – Depend on contracts

Java is especially well-suited to teaching SOLID because it has first-class support for `interface` , `abstract class` , and strict static typing — all of which make violations (and fixes) very visible.

3. Why SOLID Matters in Enterprise Applications

Suppose the business says:

"We are changing our notification provider from Email to WhatsApp."

In a poorly designed system:

```
public class Account {  
  
    public void withdraw(double amount) {  
        // ... withdrawal logic ...  
        EmailSender.send(customerEmail, "Withdrawal successful");  
    }  
}
```

Now changing notification systems requires modifying `Account`. But `Account` should not care **how** notifications are sent.

A better architecture separates concerns:

```
Account  
|  
+---- Transaction Service  
|  
+---- Notification Service  
|  
+---- Authentication Service  
|  
+---- Repository
```

That separation is the thinking behind SOLID.

S — Single Responsibility Principle

4. What is SRP?

A class should have **one reason to change**.

This does **not** mean "a class should have only one method." It means a class should represent **one cohesive responsibility**.

5. Bad Example – Multiple Responsibilities

```

public class BankAccount {

    private double balance;

    public void deposit(double amount) {
        balance += amount;
    }

    public void withdraw(double amount) {
        balance -= amount;
    }

    public void saveToDatabase() {
        // JDBC / Hibernate logic here
    }

    public void sendEmail() {
        // SMTP logic here
    }

    public String generateStatement() {
        return "Statement...";
    }

    public double calculateTax() {
        return balance * 0.1;
    }
}

```

This class handles account operations, database persistence, email communication, statement generation, and tax calculation. It has **many reasons to change**:

Database requirements change	→ BankAccount changes
Email provider changes	→ BankAccount changes
Tax rules change	→ BankAccount changes
Statement format changes	→ BankAccount changes

This violates SRP.

6. SRP-Compliant Design

```

public class BankAccount {

    private double balance;

    public void deposit(double amount) {
        balance += amount;
    }
}

```

```

    public void withdraw(double amount) {
        balance -= amount;
    }

    public double getBalance() {
        return balance;
    }
}

public class AccountRepository {

    public void save(BankAccount account) {
        System.out.println("Saving account to database");
    }
}

public class NotificationService {

    public void send(String message) {
        System.out.println("Sending notification: " + message);
    }
}

public class StatementGenerator {

    public String generate(BankAccount account) {
        return "Statement for balance: " + account.getBalance();
    }
}

```

Each class now has a single, clear responsibility.

7. SRP in Our Banking System

Our teaching-level `Account` class typically contains:

```

deposit()
withdraw()
closeAccount()
reopenAccount()
setPin()
verifyPin()

```

That's acceptable for a classroom exercise (high cohesion — all operations relate to "account state"). In a large enterprise system, we would eventually split into:

```
Account
AuthenticationService
TransactionService
AccountLifecycleService
AccountRepository
NotificationService
```

The goal is not to blindly create hundreds of classes — the goal is **high cohesion**.

O — Open/Closed Principle

8. What is OCP?

Software entities should be **open for extension** but **closed for modification**.

We should be able to add new behavior without constantly editing existing, tested code.

9. Bad Example – Growing if/else Chains

```
public class InterestCalculator {

    public double calculate(String accountType, double balance) {

        if (accountType.equals("Savings")) {
            return balance * 0.04;
        } else if (accountType.equals("Current")) {
            return balance * 0.01;
        } else if (accountType.equals("Salary")) {
            return balance * 0.05;
        }

        return 0;
    }
}
```

Tomorrow the bank introduces Fixed Deposit, NRI Account, Senior Citizen Account, Student Account... and we keep growing this `if/else` chain, risking breaking existing behavior every time.

10. OCP Solution

Define an abstraction (an `interface` in Java):

```
public interface InterestPolicy {
    double calculate(double balance);
}

public class SavingsInterestPolicy implements InterestPolicy {
    @Override
    public double calculate(double balance) {
        return balance * 0.04;
    }
}

public class CurrentInterestPolicy implements InterestPolicy {
    @Override
    public double calculate(double balance) {
        return balance * 0.01;
    }
}

public class SalaryInterestPolicy implements InterestPolicy {
    @Override
    public double calculate(double balance) {
        return balance * 0.05;
    }
}
```

Now we can add a new policy without touching existing ones:

```
public class FixedDepositInterestPolicy implements InterestPolicy {
    @Override
    public double calculate(double balance) {
        return balance * 0.07;
    }
}
```

11. Strategy Pattern Connection

OCP frequently leads toward the **Strategy Pattern**. Instead of branching on type:

```
InterestPolicy policy = new SavingsInterestPolicy();
double interest = policy.calculate(balance);
```

The algorithm now varies independently of the code that uses it — a very natural fit in Java, since `interface` was designed exactly for this.

L — Liskov Substitution Principle

12. What is LSP?

Objects of a subclass should be usable wherever objects of the superclass are expected, **without breaking correctness**.

This is one of the most commonly misunderstood SOLID principles.

13. Simple Example

```
public class Account {
    public void withdraw(double amount) { /* ... */ }
}

public class SavingsAccount extends Account {
    @Override
    public void withdraw(double amount) { /* valid override */ }
}
```

This is fine if `SavingsAccount` honors the contract of `Account`. But:

```
public class FixedDepositAccount extends Account {
    @Override
    public void withdraw(double amount) {
        throw new UnsupportedOperationException("Withdrawal is impossible");
    }
}
```

If code written against `Account` assumes `withdraw()` is always valid:

```
account.withdraw(1000);
```

replacing `Account` with `FixedDepositAccount` breaks the program. This signals the abstraction itself is wrong.

14. Classic Rectangle/Square Example

```

public class Rectangle {

    protected int width;
    protected int height;

    public void setWidth(int width) {
        this.width = width;
    }

    public void setHeight(int height) {
        this.height = height;
    }

    public int area() {
        return width * height;
    }
}

```

```

public class Square extends Rectangle {

    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width;
    }

    @Override
    public void setHeight(int height) {
        this.width = height;
        this.height = height;
    }
}

```

```

Rectangle rectangle = new Square();
rectangle.setWidth(10);
rectangle.setHeight(20);
System.out.println(rectangle.area());    // Expected 200, prints 400

```

A caller relying on `Rectangle` 's contract (independent width/height) gets a surprise. `Square` does not behave like a `Rectangle` — inheritance is inappropriate here.

15. LSP in Banking

```

public interface Withdrawable {
    void withdraw(double amount);
}

```

```

}

public class SavingsAccount extends Account implements Withdrawable {
    @Override
    public void withdraw(double amount) { /* valid */ }
}

public class CurrentAccount extends Account implements Withdrawable {
    @Override
    public void withdraw(double amount) { /* valid */ }
}

```

But `FixedDepositAccount` should **not** implement `Withdrawable` at all, since it cannot honestly satisfy the contract:

```

public class FixedDepositAccount extends Account {
    // no withdraw() – because it genuinely cannot support it
}

```

Design tree:

```

Account
├── SavingsAccount
├── CurrentAccount
└── FixedDepositAccount

Withdrawable
├── SavingsAccount
└── CurrentAccount

```

I — Interface Segregation Principle

16. What is ISP?

Clients should not be forced to depend on methods they do not use.

In Java this maps directly onto `interface` .

17. Bad Interface

```

public interface BankService {
    void deposit(double amount);
    void withdraw(double amount);
    void transfer(double amount, String toAccount);
    void printStatement();
    void applyForLoan();
}

public class ATM implements BankService {
    // forced to implement printStatement() and applyForLoan()
    // even though an ATM might not support them
}

```

The interface is too large — it forces unrelated capabilities onto every implementer.

18. ISP Solution

Break the large interface into smaller, focused ones:

```

public interface Depositable {
    void deposit(double amount);
}

public interface Withdrawable {
    void withdraw(double amount);
}

public interface Transferable {
    void transfer(double amount, String toAccount);
}

public interface StatementProvider {
    void printStatement();
}

public interface LoanEligible {
    void applyForLoan();
}

```

19. Java and Multiple Interfaces

Java allows a class to implement multiple interfaces (unlike single inheritance for classes), which is perfect for capability-based design:

```
public class SavingsAccount extends Account
    implements Depositable, Withdrawable, Transferable, StatementProvider {
    // implements exactly what it needs
}
```

```
public class FixedDepositAccount extends Account implements Depositable, StatementProvider {
    // does not implement Withdrawable – and that's correct
}
```

◀  ▶

Capability-based design instead of one giant interface.

D — Dependency Inversion Principle

20. What is DIP?

High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details. Details should depend on abstractions.

This is usually the hardest principle to internalize at first.

21. What is Tight Coupling?

```
public class Bank {

    private MySQLDatabase database = new MySQLDatabase();
}
```

`Bank` is now directly coupled to `MySQLDatabase`. Switching to PostgreSQL or MongoDB means editing `Bank` every time.

22. DIP Solution

Define an abstraction:

```
public interface AccountRepository {
    void save(Account account);
    Account findById(int accountNumber);
}
```

```

public class MySQLAccountRepository implements AccountRepository {
    @Override
    public void save(Account account) {
        System.out.println("Saving to MySQL");
    }

    @Override
    public Account findById(int accountNumber) {
        System.out.println("Fetching from MySQL");
        return null;
    }
}

public class PostgreSQLAccountRepository implements AccountRepository {
    @Override
    public void save(Account account) {
        System.out.println("Saving to PostgreSQL");
    }

    @Override
    public Account findById(int accountNumber) {
        System.out.println("Fetching from PostgreSQL");
        return null;
    }
}

public class Bank {

    private final AccountRepository repository;

    public Bank(AccountRepository repository) {
        this.repository = repository;
    }
}

```

We can now inject any implementation:

```

Bank bank1 = new Bank(new MySQLAccountRepository());
Bank bank2 = new Bank(new PostgreSQLAccountRepository());

```

Bank doesn't care which database is used. This is **Dependency Injection**, and in real Java enterprise systems this is usually done automatically by a framework like **Spring** using `@Autowired` or constructor injection.

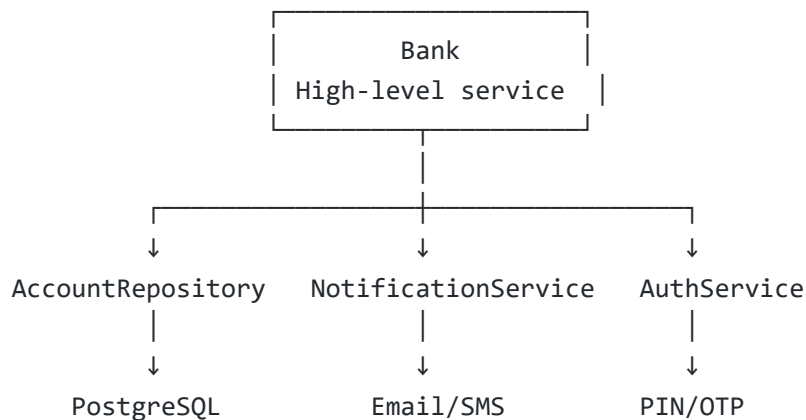
23. Dependency Inversion vs Dependency Injection

- **Dependency Inversion Principle:** the design principle — depend on abstractions.
- **Dependency Injection:** a technique to implement that principle — the dependency is supplied from outside (constructor, setter, or a framework).

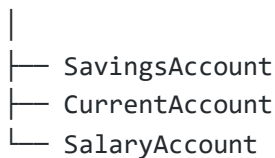
Without **DI**: **Bank** → MySQLDatabase

With **DI**: **Bank** → AccountRepository ← MySQLDatabase

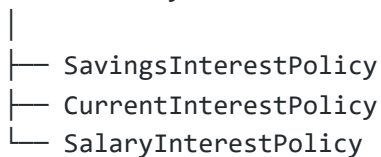
24. Putting All Five Together



Account

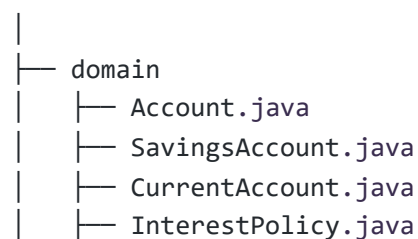


InterestPolicy



25. Enterprise Package Structure

com.bank



```

|   └─ ...
|
|   └─ service
|       ├── Bank.java
|       └─ NotificationService.java
|
|   └─ repository
|       └─ AccountRepository.java
|
|   └─ Main.java

```

For the classroom, we keep everything conceptually grouped but can demonstrate in one file/set of files.

26. Complete SOLID Bank Implementation (Java)

```

import java.util.HashMap;
import java.util.Map;
import java.util.Optional;

// =====
// ACCOUNT ABSTRACTION
// =====

public abstract class Account {

    private final int accountNumber;
    private final String name;
    private final int age;
    private double balance;
    private String status;
    private Integer pin;

    public Account(int accountNumber, String name, int age, double initialBalance) {
        this.accountNumber = accountNumber;
        this.name = name;
        this.age = Math.max(age, 18);
        this.balance = Math.max(initialBalance, getMinimumBalance());
        this.status = "Active";
        this.pin = null;
    }

    public abstract double getMinimumBalance();

    public int getAccountNumber() { return accountNumber; }
    public String getName() { return name; }
    public int getAge() { return age; }
    public double getBalance() { return balance; }

```

```
public String getStatus() { return status; }

public boolean deposit(double amount) {
    if (!status.equals("Active")) return false;
    if (amount <= 0) return false;

    balance += amount;
    return true;
}

public boolean withdraw(double amount, Integer enteredPin) {
    if (!status.equals("Active")) return false;

    if (pin != null) {
        if (enteredPin == null || !enteredPin.equals(pin)) return false;
    }

    if (amount <= 0) return false;
    if (balance - amount < getMinimumBalance()) return false;

    balance -= amount;
    return true;
}

public boolean closeAccount() {
    if (status.equals("Inactive")) return false;
    status = "Inactive";
    return true;
}

public boolean reopenAccount() {
    if (status.equals("Active")) return false;
    status = "Active";
    return true;
}

public boolean setPin(int newPin) {
    if (newPin >= 1000 && newPin <= 9999) {
        this.pin = newPin;
        return true;
    }
    return false;
}

public boolean verifyPin(int enteredPin) {
    return pin != null && pin.equals(enteredPin);
}

public boolean hasPin() {
    return pin != null;
}
}
```

```
// =====
// ACCOUNT TYPES
// =====

public class SavingsAccount extends Account {

    public SavingsAccount(int accountNumber, String name, int age, double initialBalance) {
        super(accountNumber, name, age, initialBalance);
    }

    @Override
    public double getMinimumBalance() {
        return 500.0;
    }
}

public class CurrentAccount extends Account {

    public CurrentAccount(int accountNumber, String name, int age, double initialBalance) {
        super(accountNumber, name, age, initialBalance);
    }

    @Override
    public double getMinimumBalance() {
        return 1000.0;
    }
}

// =====
// INTEREST POLICY (OCP)
// =====

public interface InterestPolicy {
    double calculate(double balance);
}

public class SavingsInterestPolicy implements InterestPolicy {
    @Override
    public double calculate(double balance) {
        return balance * 0.04;
    }
}

public class CurrentInterestPolicy implements InterestPolicy {
    @Override
    public double calculate(double balance) {
        return balance * 0.01;
    }
}

```

```
// =====
// REPOSITORY (DIP)
// =====

public interface AccountRepository {
    void save(Account account);
    Optional<Account> find(int accountNumber);
}

public class InMemoryAccountRepository implements AccountRepository {

    private final Map<Integer, Account> accounts = new HashMap<>();

    @Override
    public void save(Account account) {
        accounts.put(account.getAccountNumber(), account);
    }

    @Override
    public Optional<Account> find(int accountNumber) {
        return Optional.ofNullable(accounts.get(accountNumber));
    }
}

// =====
// NOTIFICATION (ISP + OCP + DIP)
// =====

public interface NotificationService {
    void send(String recipient, String message);
}

public class EmailNotificationService implements NotificationService {
    @Override
    public void send(String recipient, String message) {
        System.out.println("EMAIL → " + recipient + ": " + message);
    }
}

public class SMSNotificationService implements NotificationService {
    @Override
    public void send(String recipient, String message) {
        System.out.println("SMS → " + recipient + ": " + message);
    }
}

// =====
// BANK – HIGH-LEVEL SERVICE
// =====

public class Bank {
```

```

private final AccountRepository repository;
private final NotificationService notificationService;

public Bank(AccountRepository repository, NotificationService notificationService) {
    this.repository = repository;
    this.notificationService = notificationService;
}

public Account createSavingsAccount(int accountNumber, String name, int age, double initialBalance) {
    Account account = new SavingsAccount(accountNumber, name, age, initialBalance);
    repository.save(account);
    return account;
}

public Account createCurrentAccount(int accountNumber, String name, int age, double initialBalance) {
    Account account = new CurrentAccount(accountNumber, name, age, initialBalance);
    repository.save(account);
    return account;
}

public boolean deposit(int accountNumber, double amount) {
    Optional<Account> maybeAccount = repository.find(accountNumber);
    if (maybeAccount.isEmpty()) return false;

    Account account = maybeAccount.get();
    boolean success = account.deposit(amount);

    if (success) {
        notificationService.send(account.getName(), "Deposited Rs. " + amount);
    }
    return success;
}

public boolean withdraw(int accountNumber, double amount, Integer pin) {
    Optional<Account> maybeAccount = repository.find(accountNumber);
    if (maybeAccount.isEmpty()) return false;

    Account account = maybeAccount.get();
    boolean success = account.withdraw(amount, pin);

    if (success) {
        notificationService.send(account.getName(), "Withdrawn Rs. " + amount);
    }
    return success;
}
}

```

27. Using the SOLID Bank

```
AccountRepository repository = new InMemoryAccountRepository();
NotificationService notification = new EmailNotificationService();

Bank bank = new Bank(repository, notification);

Account account = bank.createSavingsAccount(101, "Ravi", 17, 200);
```

The account auto-corrects:

```
Age:      17  → 18
Balance: 200 → 500
```

```
account.setPin(1234);

bank.deposit(101, 1000);

bank.withdraw(101, 500, 1234); // succeeds
bank.withdraw(101, 500, 9999); // fails: wrong PIN → false
```

28. Changing Email to SMS

```
NotificationService notification = new EmailNotificationService();
Bank bank = new Bank(repository, notification);
```

Later:

```
NotificationService notification = new SMSNotificationService();
Bank bank = new Bank(repository, notification);
```

Bank was **not modified**. This demonstrates **OCP + DIP**.

29. Adding a New Database

```
public class PostgreSQLAccountRepository implements AccountRepository {

    @Override
    public void save(Account account) {
        System.out.println("Saving account to PostgreSQL");
    }

    @Override
    public Optional<Account> find(int accountNumber) {
```

```

        System.out.println("Searching PostgreSQL");
        return Optional.empty();
    }
}

```

```

AccountRepository repository = new PostgreSQLAccountRepository();
Bank bank = new Bank(repository, notification);

```

Again — Bank did not change.

30. A Note on Java-Specific Tools

Java gives students concrete language features that map directly onto SOLID:

Feature	Purpose
interface	Define abstractions/contracts (ISP, DIP)
abstract class	Share common state/behavior while forcing subclasses to define specifics (SRP, OCP)
implements	Fulfill a contract (can implement many interfaces — great for ISP)
extends	Inherit shared behavior (must respect LSP)
@Override	Compiler-enforced check that a subclass method matches its contract
Constructor injection	The standard way to apply DIP/DI in plain Java
Spring's @Autowired / @Service / @Repository	Framework-level DI container (mention only — not required for this course)

A key Java-specific teaching point: because Java only allows **single class inheritance** but **multiple interface implementation**, ISP and DIP become natural design defaults rather than afterthoughts — this is different from some other languages and worth calling out explicitly.

31. Where Each SOLID Principle Appears

Principle	Our Bank Implementation
S – SRP	<code>Account</code> , <code>AccountRepository</code> , <code>NotificationService</code> , <code>InterestPolicy</code> each have one responsibility
O – OCP	New account types, interest policies, and notification channels can be added without editing existing classes
L – LSP	<code>SavingsAccount</code> / <code>CurrentAccount</code> can be used anywhere an <code>Account</code> is expected
I – ISP	Small focused interfaces: <code>InterestPolicy</code> , <code>AccountRepository</code> , <code>NotificationService</code> (and <code>Depositable</code> / <code>Withdrawable</code> / <code>Transferable</code> when needed)
D – DIP	<code>Bank</code> depends on <code>AccountRepository</code> and <code>NotificationService</code> interfaces, not concrete classes

32. Coupling vs Cohesion

Coupling — how strongly one component depends on another.

Bad (high coupling):

`Bank` → `MySQLDatabase` → `EmailSender` → `TwilioClient`

Cohesion — how closely related the responsibilities inside a class are.

Good (high cohesion):

`Account`

```

├─ deposit()
├─ withdraw()
├─ closeAccount()
└─ reopenAccount()

```

Enterprise design aims for **low coupling + high cohesion**.

33. SOLID Does Not Mean "Never Modify Code"

OCP does not mean "never touch existing code." It means:

Design the system so frequently changing behavior can be extended without unnecessarily modifying stable, tested components.

Adding `WhatsAppNotificationService` implements `NotificationService` should never require editing `Bank` , because notification behavior was designed for extension from the start.

34. Trainer Demonstration: A Change Request

Business requirement: *"The bank currently sends notifications via email. Management now wants SMS too."*

Poor design

```
public class Bank {  
  
    public void withdraw(Account account, double amount) {  
        account.withdraw(amount, null);  
        EmailSender.send(account.getName(), "Withdrawal successful");  
        SmsSender.send(account.getName(), "Withdrawal successful"); // added later – Bank modi  
    }  
}
```



SOLID design

```
Bank bank = new Bank(repository, new SMSNotificationService());
```

Bank remains untouched — only the injected dependency changes.

35. A Practical Mental Model for Students

Ask yourself	Principle
"What is this class responsible for?"	S
"What happens when the business adds a new type?"	O
"Can I replace the parent with this child without breaking the program?"	L
"Does this class really need every method in this interface?"	I
"Am I depending directly on a concrete class instead of an abstraction?"	D

36. Final Takeaway

Activity 1 → Basic Java (**variables**, control flow)
Activity 2 → Classes & Objects
Activity 3 → Validation + State + Security (encapsulation)
SOLID → Designing maintainable classes
Enterprise → Services + Repositories + Interfaces → Scalable Applications

OOP teaches us how to create objects. SOLID teaches us how to organize those objects into systems that can survive change.

Good Code → Works
SOLID Code → Works + Can Change + Can Scale + Can **Be** Tested + Can **Be** Maintained

Section 1 — SRP: "The class that does everything"

Scenario: GreenLeaf Bank's `BankAccount` class currently handles deposits, withdrawals, saving to a database, sending emails, and printing statements — all in one file. Your manager says: *"Every time we touch email, we accidentally break withdrawals. Fix it before we add anything new."*

1. **(Warm-up)** Open the given messy `BankAccount` class. In comments, list every distinct "reason to change" you can find (e.g., "changes if email provider changes", "changes if DB changes"). You should find at least 4.
2. **(Warm-up)** Write a plain-English one-line "job description" for what `BankAccount` *should* be responsible for, if it only did account operations (deposit/withdraw/balance).
3. **(Core)** Extract `AccountRepository` (a class with `save(BankAccount account)`, printing "Saving account to database") and `NotificationService` (a class with `send(String message)`, printing "Sending: ..."). Remove all database and email code from `BankAccount`.
4. **(Core)** Extract a `StatementGenerator` class with a `generate(BankAccount account)` method that returns a formatted string. Write a small `Main` class that creates an account, deposits/withdraws a couple of times, then uses `AccountRepository`, `NotificationService`, and `StatementGenerator` together to prove the split works.
5. **(Wrap-up)** In 3–4 sentences: how many classes did you end up with, and why is this easier to test than the original single class? (Answer without running any tests — just reason about it.)

End-of-lab checkpoint: `BankAccount` now only has `deposit()`, `withdraw()`, and simple getters. Nothing else.

Section 2 — OCP: "New account types keep arriving"

Scenario: Starting from your Lab 1 solution, GreenLeaf Bank's product team announces: *"We're launching Current Accounts and Salary Accounts this month, and probably 2–3 more account types next*

quarter. Don't make us rewrite the interest calculator every time."

1. **(Warm-up)** Look at (or write) an `InterestCalculator` with an `if/else` chain based on a `String` `accountType`. Predict, in comments, what code you'd have to touch to add a 4th account type — list every line/method affected.
2. **(Warm-up)** Define the `InterestPolicy` interface with one method: `double calculate(double balance)`.
3. **(Core)** Implement `SavingsInterestPolicy` (4%) and `CurrentInterestPolicy` (1%) as separate classes implementing `InterestPolicy`. Update `Main` to compute interest using `policy.calculate(balance)` instead of the `if/else` chain.
4. **(Core)** GreenLeaf now wants **Salary Accounts** (5% interest). Add `SalaryAccount` extends `Account` (reuse your `Account` abstraction — if you don't have one yet, use `BankAccount` from Lab 1) and `SalaryInterestPolicy`, **without editing any existing policy class**. Also make `Bank / Main` accept a `NotificationService` so you can add a second one (e.g. `SMSNotificationService`) later without touching `Bank`'s internals — just swap what's passed to the constructor.
5. **(Wrap-up)** Diff mentally: for the Salary Account requirement, which existing files did you have to open and edit vs. which files were brand new? Write one sentence confirming zero existing policy classes changed.

End-of-lab checkpoint: Adding a new account type/interest rate never requires editing an existing policy class.

Section 3 — LSP: "The Fixed Deposit account breaks things"

Scenario: A teammate excitedly adds `FixedDepositAccount` extends `Account` and overrides `withdraw()` to throw an exception, since FDs can't be withdrawn early. QA reports: *"Our automated test loops over a list of accounts and calls `withdraw()` on each — it crashes on Fixed Deposits!"* Your manager asks you to investigate and fix the design.

1. **(Warm-up)** Type out the classic `Rectangle / Square` example from the handbook. Run it: confirm you get 400 instead of the expected 200 when calling `setWidth(10)` then `setHeight(20)` on a `Square` stored as a `Rectangle`.
2. **(Warm-up)** In 2–3 sentences, explain *why* the `Square/Rectangle` example breaks LSP — what assumption does calling code make about `Rectangle` that `Square` violates?
3. **(Core)** Create `FixedDepositAccount` extends `Account` that overrides `withdraw()` to throw `UnsupportedOperationException`. Write a small loop that calls `withdraw()` on a `List<Account>` containing a `SavingsAccount` and a `FixedDepositAccount`, and observe/report the crash.
4. **(Core)** Fix the design: introduce a `Withdrawable` interface with `withdraw(double amount)`. Make only `SavingsAccount` and `CurrentAccount` implement it. Rewrite your loop to iterate over `List<Withdrawable>` instead of `List<Account>` for withdrawal operations, so `FixedDepositAccount` is never asked to do something it can't honestly do.

5. **(Wrap-up)** Write 3–4 sentences: why is "make `FixedDepositAccount` implement `Withdrawable` and just throw an exception" the *wrong* fix, even though it compiles? Reference the LSP substitution rule directly.

End-of-lab checkpoint: No class in the codebase overrides a method just to throw "not supported" — types only implement contracts they can truly fulfill.

Section 4 — ISP + DIP: "The ATM team complains, then the database changes"

Scenario: Two things land on your desk in the same week. First, the ATM hardware team says: "*We only need deposit and withdraw — why does your `BankService` interface force us to implement `applyForLoan()` and `printStatement()` ?*" Then, right after you fix that, your manager says: "*Great news — we're migrating from our in-memory storage to a real file-based store next sprint. Make sure `Bank` doesn't need to change when that happens.*"

1. **(Warm-up)** Given a fat interface `BankService` (`deposit` , `withdraw` , `transfer` , `printStatement` , `applyForLoan`), write (in comments) the `ATM` class that implements it and mark which methods it's forced to implement but doesn't really need.
2. **(Core — ISP)** Split `BankService` into `Depositable` , `Withdrawable` (reuse Lab 3's), `Transferable` , `StatementProvider` , and `LoanEligible` . Rewrite `ATM` implements `Depositable`, `Withdrawable` only, and `SavingsAccount` implements `Depositable`, `Withdrawable`, `Transferable`, `StatementProvider` . Confirm it compiles.
3. **(Warm-up — DIP)** In your current `Bank` class, find the line(s) where it directly creates a concrete class (e.g. `new InMemoryAccountRepository()` typed as the concrete class, not the interface). Rewrite `Bank` 's field and constructor parameter types to use the `AccountRepository` interface instead of the concrete class.
4. **(Core — DIP)** Implement `FileAccountRepository` implements `AccountRepository` that writes/reads account data to/from a plain text file (one line per account is fine — keep it simple: `accountNumber,name,balance`). Swap it into `Main` by changing only the single line that constructs the repository — prove `Bank.java` itself needs zero edits.
5. **(Wrap-up)** Write a short paragraph (4–6 sentences) reflecting on the whole 4-lab journey: pick one specific new requirement (e.g., "add a `MongoAccountRepository` ", or "add a `WhatsAppNotificationService` ") and explain exactly which files you'd create vs. touch today — and contrast that with how many files you'd have had to touch in the **original single `BankAccount` class from Lab 1**.

End-of-lab checkpoint: `Bank` depends only on interfaces (`AccountRepository` , `NotificationService`); every class implements only the capability-interfaces it genuinely needs.